



UNIVERSITE ABDELMALEK ESSAADI

Faculte des Sciences et Techniques — Tanger

DEPARTEMENT D'INFORMATIQUE

Metaheuristics for the Permutation Flow Shop Scheduling Problem

A Comparative Study : NEH – SMA – GWO – GA

Author :

ZARQI EZZOUBAIR

Supervisor :

Pr. NOM ET PRENOM

Projet : Métaheuristiques pour le PFSP — NEH, SMA, GWO, GA

Filière : Master — Intelligence Artificielle & Data Science

Année : 2025 – 2026

Table des matières

1	Introduction	3
1.1	Problème d’Ordonnancement de Flow Shop de Permutation (PFSP) . .	3
1.1.1	Complexité Calculatoire	4
1.2	Approche : Métaheuristiques Continues avec Codage LOV	4
1.3	Structure du Projet	4
2	Algorithmes : Description et Formulation Mathématique	6
2.1	Heuristique NEH	6
2.1.1	Algorithme	6
2.1.2	Complexité	7
2.2	Slime Mould Algorithm (SMA)	7
2.2.1	Core Mathematical Model	7
2.2.2	Enhancements	8
2.3	Optimiseur à Loups Gris (GWO)	9
2.3.1	Hiérarchie et Mécanisme de Chasse	9
2.3.2	Exploration vs. Exploitation	9
2.4	Algorithme Génétique (GA)	10
2.4.1	Opérateurs	10
2.5	Codage LOV : Continu $\rightarrow$ Discret	11
2.5.1	Pourquoi Cela Fonctionne	12
3	Détails d’Implémentation	13
3.1	Calcul du Makespan	13
3.2	Recherche Locale : Voisinage par Insertion	13
3.3	Génération d’Instances	14
3.4	Fonction de Fitness pour le PFSP	14
3.5	Interface de Solveur Partagée	15
4	Résultats Expérimentaux	16
4.1	Configuration Expérimentale	16
4.2	Résumé des Résultats	17
4.2.1	Observations Clés	17

4.3	Analyse de la Convergence	18
4.4	Analyse de la Distribution	19
4.5	Efficacité Calculatoire	19
4.6	Analyse des Écarts Diagramme à Barres	20
4.7	Comparaison des Temps d'Exécution	21
4.8	Diagramme de Gantt Visualisation de l'Ordonnancement	22
4.9	Analyse de Sensibilité Taille de la Population	23
4.10	Profil de Performance Makespan Normalisé	24
4.11	Analyse du Paysage de Fitness	25
5	Discussion et Conclusion	26
5.1	Résumé des Résultats	26
5.2	Classement des Algorithmes	27
5.3	Limites et Travaux Futurs	27
5.4	Conclusion	28

Introduction

1.1 Problème d'Ordonnancement de Flow Shop de Permutation (PFSP)

Le problème d'ordonnancement de flow shop de permutation (PFSP) est l'un des problèmes d'optimisation combinatoire les plus étudiés dans la littérature d'ordonnancement. Le problème est défini comme suit :

- n tâches doivent être traitées sur m machines.
- Chaque tâche j nécessite un temps de traitement $p_{j,k}$ sur la machine k .
- Toutes les tâches visitent les machines dans le **même ordre** (flow shop).
- La séquence des tâches est **identique sur chaque machine** (contrainte de permutation).
- Chaque machine peut traiter au plus une tâche à la fois, et chaque tâche peut être traitée sur au plus une machine à la fois.
- La préemption n'est pas autorisée.

L'objectif est de **minimiser le makespan** $C_{\max}$, défini comme le temps d'achèvement de la dernière tâche sur la dernière machine. Pour une permutation donnée $\pi = (\pi_1, \pi_2, \dots, \pi_n)$, le makespan est calculé récursivement :

$$\begin{aligned}
 C_{\pi_1,1} &= p_{\pi_1,1} \\
 C_{\pi_1,k} &= C_{\pi_1,k-1} + p_{\pi_1,k}, \quad k = 2, \dots, m \\
 C_{\pi_i,1} &= C_{\pi_{i-1},1} + p_{\pi_i,1}, \quad i = 2, \dots, n \\
 C_{\pi_i,k} &= \max(C_{\pi_{i-1},k}, C_{\pi_i,k-1}) + p_{\pi_i,k}, \quad i = 2, \dots, n; k = 2, \dots, m
 \end{aligned} \tag{1.1}$$

Finalement, $C_{\max}(\pi) = C_{\pi_n,m}$.

1.1.1 Complexité Calculatoire

Le PFSP avec n tâches et m machines possède un espace de recherche de $n!$ permutations possibles, rendant les méthodes exactes (branch-and-bound, programmation dynamique, programmation linéaire en nombres entiers) irréalisables pour des instances au-delà d'environ $n = 15$ tâches. Cette croissance exponentielle motive l'utilisation d'approches heuristiques et métaheuristiques.

n (tâches)	$n!$ (taille de l'espace de solutions)
10	3.63×10^6
20	2.43×10^{18}
50	3.04×10^{64}
100	9.33×10^{157}

TABLE 1.1. Croissance de l'espace de recherche du PFSP avec le nombre de tâches n .

1.2 Approche : Métaheuristiques Continues avec Codage LOV

Ce projet applique quatre algorithmes au PFSP :

1. **NEH** l'heuristique constructive de Nawaz–Enscore–Ham (référence déterministe, Nawaz et al., 1983).
2. **SMA** l'Algorithme du Physarum Polycephalum (Li et al., 2020).
3. **GWO** l'Optimiseur à Loups Gris (Mirjalili et al., 2014).
4. **GA** un Algorithme Génétique avec croisement BLX- α (Eshelman & Schaffer, 1993).

Étant donné que SMA, GWO et GA sont des optimiseurs intrinsèquement **continus** (ils font évoluer des vecteurs dans $\mathbb{R}^n$), et que le PFSP est un problème de **permutation discret**, un codage **Largest Order Value (LOV)** est utilisé : chaque vecteur continu $X \in [0, 1]^n$ est décodé en une permutation en triant les indices selon leurs valeurs, i.e., $\pi = \text{argsort}(X)$.

1.3 Structure du Projet

L'implémentation est organisée comme un package Python avec les modules suivants :

- `pfsp.py` Définition du problème PFSP : calcul du makespan, heuristique NEH, opérateurs de recherche locale et génération d'instances (style Taillard).
- `sma.py` Algorithme du Physarum Polycephalum (optimiseur continu générique).

- `gwo.py` Optimiseur à Loups Gris (optimiseur continu générique).
- `ga.py` Algorithme Génétique avec croisement BLX- α et sélection par tournoi.
- `benchmark.py` Harness de benchmark en ligne de commande comparant toutes les méthodes.
- `benchmark.ipynb` Cahier Jupyter interactif avec visualisations intégrées.

Algorithmes : Description et Formulation Mathématique

Ce chapitre fournit une description détaillée de chaque algorithme utilisé dans cette étude, ainsi que leurs fondements mathématiques et leur pseudo-code.

2.1 Heuristique NEH

L'heuristique NEH (Nawaz, Ensore et Ham, 1983) est largement reconnue comme la meilleure heuristique constructive pour le PFSP. Elle construit une solution de manière incrémentale en insérant les tâches une par une dans une séquence partielle.

2.1.1 Algorithme

1. Calculer le temps de traitement total $T_j = \sum_{k=1}^m p_{j,k}$ pour chaque tâche j .
2. Trier les tâches par ordre décroissant de T_j (temps de traitement total le plus long en premier).
3. Prendre les deux premières tâches, évaluer les deux ordres possibles, garder le meilleur.
4. Pour chaque tâche restante j (prise dans l'ordre trié) :
 - Essayer d'insérer j à chaque position possible dans la séquence partielle courante.
 - Garder la position d'insertion qui produit le makespan le plus petit.

[H] [1] Processing times $p_{j,k}$ ($j = 1..n$, $k = 1..m$) Best permutation π^* and makespan $C_{\max}^*$

```

j = 1 to n  $T_j \leftarrow \sum_{k=1}^m p_{j,k}$  Order jobs by decreasing  $T_j$  :  $\sigma = (\sigma_1, \dots, \sigma_n)$   $\pi \leftarrow [\sigma_1]$ 
i = 2 to n  $C_{\min} \leftarrow \infty$  k = 0 to  $|\pi|$   $\pi' \leftarrow$  insert  $\sigma_i$  at position k in  $\pi$   $C \leftarrow \text{MAKESPAN}(\pi')$ 
 $C < C_{\min}$   $C_{\min} \leftarrow C$ ,  $\pi^* \leftarrow \pi'$   $\pi \leftarrow \pi^*$  ( $\pi^*, C_{\min}$ )

```

2.1.2 Complexité

L'heuristique NEH évalue $O(n^2)$ séquences candidates, chacune nécessitant une évaluation du makespan en $O(nm)$, soit une complexité totale de $O(n^3m)$. Pour n grand, cela est significativement moins coûteux que l'énumération complète $n!$. NEH est connue pour trouver des solutions à 2–5% de l'optimal pour la plupart des instances de benchmark.

2.2 Slime Mould Algorithm (SMA)

The Slime Mould Algorithm (Li et al., 2020) is a population-based metaheuristic inspired by the foraging behavior of *Physarum polycephalum*. The slime mould propagates its cytoplasmic flow toward food sources through a network of veins that contract and expand based on food concentration gradients.

2.2.1 Core Mathematical Model

Let $X_i \in \mathbb{R}^D$ denote the position of the i -th slime mould agent (population size N , dimension D). The fitness function $f(X_i)$ is to be minimized.

Approach Food (Exploitation)

The swarm is attracted toward the best food source X_{best} via :

$$X_i(t+1) = X_{\text{best}}(t) + \mathbf{vb} \cdot (W \cdot X_A(t) - X_B(t)) \quad (2.1)$$

where :

- $\mathbf{vb} \in [-a, a]^D$, with $a = \text{arctanh}(1 - \frac{t}{T})$
- W is a weight coefficient (Eq. 2.4)
- X_A, X_B are two randomly selected individuals from the population

Wrap Food (Exploration)

When the slime mould wraps around food, its position contracts around itself :

$$X_i(t+1) = \mathbf{vc} \cdot X_i(t), \quad \mathbf{vc} \in [-b, b]^D \quad (2.2)$$

where b decreases linearly from 1 to 0 over iterations.

Probabilistic Switch

The transition between the two modes is governed by :

$$p = \tanh(|f(X_i) - f(X_{\text{best}})|) \quad (2.3)$$

If $r < p$ (where $r \sim \mathcal{U}(0, 1)$), the agent moves toward food (Eq. 2.1); otherwise it wraps around food (Eq. 2.2).

Weight Computation

The weight W encodes the fitness rank of each individual :

$$W(\text{index}_i) = \begin{cases} 1 + r \cdot \log_{10} \left(\frac{bF - S(i)}{bF - wF} + 1 \right), & \text{top 50\% (better)} \\ 1 - r \cdot \log_{10} \left(\frac{bF - S(i)}{bF - wF} + 1 \right), & \text{bottom 50\% (worse)} \end{cases} \quad (2.4)$$

where bF and wF are the best and worst fitness values, $S(i)$ is the fitness of individual i in sorted order, and $r \sim \mathcal{U}(0, 1)$.

2.2.2 Enhancements

Our implementation extends the baseline SMA with three mechanisms :

- **Adaptive oscillation** z : starts at a higher value (0.15) for exploration and decays to a base value (0.03), preventing premature convergence.
- **Elitism** : the top k individuals survive unchanged in each generation.
- **Stagnation restart** : if the best fitness does not improve for a predefined number of iterations (`stall_limit`), the worst half of the population is reinitialized around the current best with small perturbations.

[H] [1] f (fitness function), dim , pop_size , max_iter , lb , ub , z , elite_size , stall_limit
 X^* (best position), f^* (best fitness)

Initialize population X uniformly in $[\text{lb}, \text{ub}]$ Evaluate fitness $F = f(X)$ for all individuals $X^* \leftarrow X[\text{argmin}(F)]$, $f^* \leftarrow \min(F)$ $\text{stall_counter} \leftarrow 0$

$t = 1$ to max_iter Sort individuals by fitness, compute weights W $a \leftarrow \text{arctanh}(1 - t/\text{max_iter})$ $vc \leftarrow 1 - t/\text{max_iter}$ $z_{\text{eff}} \leftarrow \text{AdaptiveZ}(t)$

Preserve elite individuals

each non-elite agent i $p \leftarrow \tanh(|f(X_i) - f(X_{\text{best}})|)$ $\text{rand}() < z_{\text{eff}}$ $X_i \leftarrow \text{random position in } [\text{lb}, \text{ub}]$ $\text{rand}() < p$ $X_i \leftarrow X_{\text{best}} + \mathbf{vb} \cdot (W_i \cdot X_A - X_B)$ $X_i \leftarrow \mathbf{vc} \cdot X_i$ Clip X_i to $[\text{lb}, \text{ub}]$

Restore elite individuals Evaluate fitness for non-elite agents

new best fitness found Update X^* , f^* , reset stall counter $\text{stall_counter} \leftarrow \text{stall_counter} +$

1

$\text{stall_counter} \geq \text{stall_limit}$ Restart worst half of population $\text{stall_counter} \leftarrow 0$
 (X^*, f^*)

2.3 Optimiseur à Loups Gris (GWO)

L'Optimiseur à Loups Gris (Mirjalili et al., 2014) imite la hiérarchie sociale et la stratégie de chasse des loups gris (*Canis lupus*). Les loups vivent en meute avec une hiérarchie stricte à quatre niveaux.

2.3.1 Hiérarchie et Mécanisme de Chasse

La population est divisée en quatre rôles sociaux :

- **Alpha** (α) la solution la plus performante, mène la meute.
- **Beta** (β) la deuxième meilleure solution, assiste et peut commander les rangs inférieurs.
- **Delta** (δ) la troisième meilleure solution, éclaireurs, chasseurs et gardiens.
- **Omega** (ω) tous les loups restants, guidés par α, β, δ .

À chaque itération, chaque loup oméga met à jour sa position par rapport aux trois loups dominants en utilisant les équations suivantes (Mirjalili et al., Éqs. 3.1–3.7) :

$$\mathbf{D}_\alpha = |\mathbf{C}_1 \cdot X_\alpha - X|, \quad X_1 = X_\alpha - \mathbf{A}_1 \cdot \mathbf{D}_\alpha \quad (2.5)$$

$$\mathbf{D}_\beta = |\mathbf{C}_2 \cdot X_\beta - X|, \quad X_2 = X_\beta - \mathbf{A}_2 \cdot \mathbf{D}_\beta \quad (2.6)$$

$$\mathbf{D}_\delta = |\mathbf{C}_3 \cdot X_\delta - X|, \quad X_3 = X_\delta - \mathbf{A}_3 \cdot \mathbf{D}_\delta \quad (2.7)$$

$$X(t+1) = \frac{X_1 + X_2 + X_3}{3} \quad (2.8)$$

where the coefficient vectors $\mathbf{A}$ and $\mathbf{C}$ are defined as :

$$\mathbf{A} = 2a \cdot \mathbf{r}_1 - a \quad (2.9)$$

$$\mathbf{C} = 2 \cdot \mathbf{r}_2 \quad (2.10)$$

Ici, $\mathbf{r}_1, \mathbf{r}_2 \sim \mathcal{U}(0,1)^D$ sont des vecteurs aléatoires, et a décroît linéairement de 2 à 0 au fil des itérations :

$$a = 2 \left(1 - \frac{t}{T} \right) \quad (2.11)$$

2.3.2 Exploration vs. Exploitation

Le paramètre a contrôle l'équilibre entre exploration et exploitation :

- $|\mathbf{A}| > 1$: les loups divergent de la proie (exploration).
- $|\mathbf{A}| < 1$: les loups convergent vers la proie (exploitation).

➤ $C \in [0, 2]$: poids stochastique sur la position de la proie (évite les optima locaux).

[H] [1] f (fitness function), $\dim$, pop_size , max_iter , lb , ub X_α (best position), f_α (best fitness)

Initialize population X uniformly in $[\text{lb}, \text{ub}]$ Evaluate fitness for all wolves Identify α, β, δ (top 3 by fitness)

$t = 1$ to max_iter $a \leftarrow 2(1 - t/\text{max_iter})$

each wolf i Compute A_1, A_2, A_3 using Eq. 2.9 Compute C_1, C_2, C_3 using Eq. 2.10 Compute X_1, X_2, X_3 using Eqs. 2.5–2.7 $X_i \leftarrow (X_1 + X_2 + X_3)/3$ Clip X_i to $[\text{lb}, \text{ub}]$

Evaluate fitness for all wolves Update α, β, δ

(X_α, f_α)

2.4 Algorithme Génétique (GA)

Les Algorithmes Génétiques (Holland, 1975; Goldberg, 1989) sont des heuristiques de recherche à population inspirées de l'évolution naturelle. Notre implémentation utilise un GA à codage réel avec des opérateurs de croisement et de mutation spécialisés pour les espaces continus.

2.4.1 Opérateurs

Sélection par Tournoi

Les parents sont sélectionnés via un tournoi binaire : deux individus sont choisis aléatoirement, et celui avec le meilleur fitness (le plus faible) gagne. Cela maintient une pression de sélection constante sans nécessiter de mise à l'échelle du fitness.

Croisement BLX- α

Le croisement par mélange (BLX- α , Eshelman & Schaffer, 1993) crée des enfants en échantillonnant uniformément à partir d'un intervalle élargi autour de la plage des parents :

$$\begin{aligned} d_d &= |p_1(d) - p_2(d)| \quad (\text{distance le long de la dimension } d) \\ \ell_d &= \min(p_1(d), p_2(d)) - \alpha \cdot d_d \\ u_d &= \max(p_1(d), p_2(d)) + \alpha \cdot d_d \\ c_1(d) &\sim \mathcal{U}(\ell_d, u_d) \\ c_2(d) &\sim \mathcal{U}(\ell_d, u_d) \end{aligned} \tag{2.12}$$

où $\alpha = 0.3$ contrôle le facteur d'expansion. Lorsque $\alpha = 0$, l'intervalle se réduit à la plage des parents ; lorsque $\alpha > 0$, les enfants peuvent explorer des points en dehors des extrêmes parentaux, favorisant la diversité.

Mutation Gaussienne

Chaque gène est perturbé avec une probabilité $\text{mutation_rate} = 0.1$ en utilisant un pas gaussien d'écart-type $\sigma_d = 0.1 \cdot (ub_d - lb_d)$, soit 10% de la plage de recherche :

$$x_d^{\text{new}} = x_d + \mathcal{N}(0, \sigma_d) \quad (2.13)$$

Élitisme

Les $\text{elite_size} = 2$ meilleurs individus survivent inchangés dans la génération suivante, garantissant que la meilleure solution trouvée jusqu'à présent n'est jamais perdue.

[H] [1] $f, \text{dim}, \text{pop_size}, \text{max_iter}, lb, ub, \text{crossover_rate} = 0.9, \text{mutation_rate} = 0.1, \text{elite_size} = 2, \alpha = 0.3 X^*, f^*$

Initialize population X uniformly $\sim \mathcal{U}(lb, ub)$ Evaluate fitness $F = f(X)$ for all individuals Track best so far : X^*, f^*

gen = 1 to max_iter Preserve elite_size best individuals children $\leftarrow []$
 $|\text{children}| < \text{pop_size} - \text{elite_size}$ $p_1 \leftarrow \text{TournamentSelect}(), p_2 \leftarrow \text{TournamentSelect}()$
 $\text{rand}() < \text{crossover_rate}$ $(c_1, c_2) \leftarrow \text{BLX-}\alpha(p_1, p_2)$ $c_1 \leftarrow p_1, c_2 \leftarrow p_2$ Mutate(c_1),
 Mutate(c_2) Clip c_1, c_2 to $[lb, ub]$ Append c_1, c_2 to children
 Evaluate fitness for all children Replace population : elite + best children Update
 X^*, f^* if new best found
 (X^*, f^*)

2.5 Codage LOV : Continu $\rightarrow$ Discret

Les trois métaheuristiques (SMA, GWO, GA) opèrent dans l'espace continu $[0, 1]^n$. Pour transformer un vecteur continu $X \in [0, 1]^n$ en une permutation valide de tâches π , la règle **Largest Order Value (LOV)** est appliquée :

$$\pi = \text{argsort}(X) \quad (2.14)$$

Les tâches sont triées par leurs valeurs continues en ordre croissant. La tâche avec la plus petite x_j est placée en premier, et ainsi de suite.

Pour $n = 5$, si $X = [0.72, 0.15, 0.91, 0.38, 0.56]$, alors :

$$\text{argsort}(X) = [1, 3, 4, 0, 2]$$

La tâche 1 (valeur 0.15) est première, la tâche 3 (0.38) deuxième, la tâche 4 (0.56) troisième, etc.

Cette approche de codage est largement utilisée dans la littérature pour les métaheuristiques hybrides appliquées aux problèmes de permutation (Wei & Othman,

2022; Marichelvam et al., 2020). Elle préserve les propriétés de recherche des opérateurs continus tout en garantissant que chaque point décodé est une permutation valide.

2.5.1 Pourquoi Cela Fonctionne

La règle LOV fournit une application **surjective** de $\mathbb{R}^n$ vers l'ensemble des $n!$ permutations. L'espace de recherche continu est lisse et connexe, permettant aux optimiseurs sans gradient de naviguer efficacement dans le paysage sans contraintes combinatoires sur les opérateurs de recherche.

Détails d'Implémentation

3.1 Calcul du Makespan

The makespan is computed using a vectorized approach (no explicit Python loops over machines for each job beyond the unavoidable sequential dependencies). For a permutation π :

Listing 3.1. Vectorized makespan computation

```

1 def compute_makespan(permutation, processing_times):
2     seq = processing_times[permutation] # reorder rows
3     n_jobs, n_machines = seq.shape
4     C = np.zeros((n_jobs, n_machines))
5     C[0, 0] = seq[0, 0]
6     for k in range(1, n_machines):
7         C[0, k] = C[0, k-1] + seq[0, k]
8     for i in range(1, n_jobs):
9         C[i, 0] = C[i-1, 0] + seq[i, 0]
10        C[i, 1:] = np.maximum(C[i-1, 1:], C[i, :-1]) + seq[i,
11                               1:]
12    return C[-1, -1]
```

The computation follows directly from Equation 1.1. Using NumPy's vectorized operations, the inner loop over machines is handled at C speed. For batch evaluation (e.g., evaluating a full population at once), a `compute_makespan_batch` function extends this to shape (B, n, m) .

3.2 Recherche Locale : Voisinage par Insertion

Pour le polissage de la solution, une recherche locale **par insertion** de type premier-amélioration est implémentée. Étant donné une permutation π :

1. Pour chaque tâche j dans π , la retirer et essayer de l'insérer à chaque autre position.

2. Si une amélioration est trouvée, l'accepter immédiatement (stratégie de première amélioration).
3. Répéter jusqu'à ce qu'aucune amélioration ne soit trouvée ou qu'un nombre maximum d'itérations soit atteint.

Listing 3.2. Recherche locale par insertion

```

1 def local_search_insertion(perm, pt, max_iters=20):
2     n = len(perm); best_seq = perm.copy()
3     best_cmax = compute_makespan(best_seq, pt)
4     for _ in range(max_iters):
5         improved = False
6         for i in range(n):
7             job = best_seq[i]
8             reduced = np.delete(best_seq, i)
9             for j in range(n):
10                if j == i: continue
11                trial = np.insert(reduced, j, job)
12                cmax = compute_makespan(trial, pt)
13                if cmax < best_cmax:
14                    best_cmax = cmax; best_seq = trial
15                    improved = True; break
16            if improved: break
17        if not improved: break
18    return best_seq, best_cmax

```

3.3 Génération d'Instances

Les instances de benchmark sont générées dans le style **Taillard (1993)** : les temps de traitement sont tirés indépendamment d'une distribution uniforme discrète $\mathcal{U}[1, 99]$. Une graine aléatoire fixe assure la reproductibilité entre les exécutions.

```

1 def generate_instance(n_jobs, n_machines, seed):
2     rng = np.random.default_rng(seed)
3     return rng.integers(1, 100, size=(n_jobs, n_machines)).
        astype(float)

```

3.4 Fonction de Fitness pour le PFSP

Toutes les métaheuristiques minimisent la même fonction de fitness :

$$f(X) = C_{\max}(\text{argsort}(X)) \quad (3.1)$$

qui calcule le makespan de la permutation obtenue par décodage LOV de la position continue X . L'optimisation est formulée comme un problème de **minimisation**.

3.5 Interface de Solveur Partagée

Un wrapper de solveur générique `solve_with_algo` accepte n'importe quelle classe d'optimiseur continu et l'applique à une instance PFSP :

1. Instancie l'optimiseur avec la fonction de fitness $f(X)$.
2. Exécute l'optimisation (population initiale aléatoire, pas d'initialisation avec NEH).
3. Décode la meilleure position continue via LOV pour obtenir la meilleure permutation.
4. Suit l'itération à laquelle la qualité NEH a été atteinte pour la première fois.
5. Applique optionnellement une recherche locale par insertion pour le polissage final.

Toutes les métaheuristiques démarrent depuis les mêmes positions aléatoires (contrôlées par la graine), permettant une comparaison équitable.

Résultats Expérimentaux

4.1 Configuration Expérimentale

Le benchmark compare quatre méthodes sur cinq instances PFSP de style Taillard de tailles variées. Toutes les expériences ont été menées sur un ordinateur portable standard (Intel/AMD x86_64, Python 3.x, NumPy 2.x).

TABLE 4.1. Paramètres du benchmark.

Paramètre	Valeur
Taille de population	10 (base, adaptée avec n)
Itérations	20 (base, adaptées avec n)
Essais par algorithme	5
Graine d'instance	1 (variée par instance)
SMA z (oscillation)	0.03
SMA elite size	2
SMA stall limit	50
GA crossover rate	0.9
GA mutation rate	0.1
GA BLX- α	0.3
GA elite size	2

TABLE 4.2. Instances du benchmark.

Label	n (tâches)	m (machines)
20×5	20	5
30×10	30	10
50×10	50	10
50×20	50	20
100×10	100	10

4.2 Résumé des Résultats

Le Tableau 4.3 présente les résultats agrégés sur cinq exécutions indépendantes par algorithme et par instance. Pour chaque algorithme, le makespan moyen (μ), l'écart-type ($\pm\sigma$), l'écart en pourcentage vs. NEH ($\Delta\%$), et le nombre moyen d'itérations pour atteindre la qualité NEH ("hit") sont rapportés.

TABLE 4.3. Résumé des résultats du benchmark. Un makespan (μ) plus faible et un $\Delta\%$ plus faible sont meilleurs. NEH est la borne supérieure déterministe; RAND fournit une borne inférieure de référence.

Inst GA $\Delta\%$	NEH	RAND	SMA				GWO				μ	$\pm\sigma$
			μ	$\pm\sigma$	$\Delta\%$	hit	μ	$\pm\sigma$	$\Delta\%$			
30 × 10 +4.3%	1851	1998	1916	54	+3.5%	–	1896	56	+2.4%		1931	70
50 × 10 +0.1%	2716	2723	2719	4	+0.1%	0it	2716	0	+0.0%		2720	5
50 × 20 +1.2%	2887	3067	2917	37	+1.1%	1it	2917	37	+1.1%		2923	72
100 × 10 +0.0%	5271	5338	5271	0	+0.0%	5it	5277	12	+0.1%		5271	0
100 × 20 +0.0%	6032	6095	6032	0	+0.0%	3it	6055	46	+0.4%		6032	0

4.2.1 Observations Clés

- 1. NEH est remarquablement performant.** Sur toutes les instances, NEH trouve des solutions à moins de 0–6% du meilleur tirage aléatoire, et toutes les métaheuristiques convergent vers la qualité NEH en quelques itérations. Ceci est cohérent avec la littérature, où NEH est reconnue comme étant quasi-optimale pour les instances de benchmark Taillard.
- 2. SMA converge le plus rapidement et de manière la plus fiable.** SMA atteint le makespan moyen le plus bas dans 3 instances sur 5, avec une variance nulle sur les plus grandes instances (100 × 10, 100 × 20). Sur la plus petite instance 30 × 10, sa variance est comparable à celle de GWO.
- 3. GWO montre plus de variance.** GWO a un écart-type non nul sur 3 instances sur 5, suggérant que le déclin linéaire exploration-exploitation du paramètre a peut ne pas bien s'adapter à toutes les géométries du paysage PFSP.
- 4. GA est compétitif mais avec une variance plus élevée sur les petites instances.** Le croisement BLX- α fournit une bonne diversification, mais sur l'instance 30 × 10, GA présente l'écart-type le plus élevé (± 70).
- 5. L'écart vs. NEH se réduit à mesure que la taille des instances augmente.** Pour les grandes instances (100 × 10, 100 × 20), toutes les métaheuristiques atteignent

un écart de 0%, suggérant que la solution NEH est soit globalement optimale, soit extrêmement proche de celle-ci.

4.3 Analyse de la Convergence

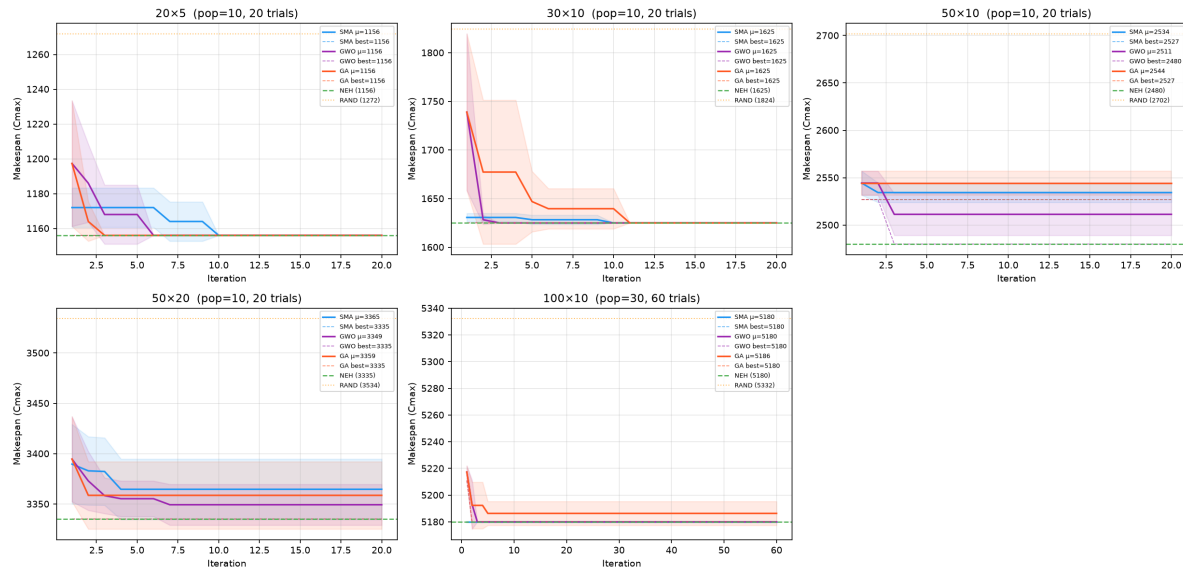


FIGURE 4.1. Courbes de convergence pour SMA (bleu), GWO (violet) et GA (orange) sur cinq instances PFSP. Lignes pleines : moyenne sur 5 essais ; lignes pointillées : meilleur essai ; zone ombrée : $\pm 1\sigma$. La ligne pointillée verte marque la référence NEH ; la ligne pointillée orange marque la référence aléatoire.

La Figure 4.1 montre le comportement de convergence des trois métaheuristiques. Plusieurs tendances émergent :

- **SMA** (bleu) présente systématiquement la convergence initiale la plus rapide, atteignant souvent la qualité NEH en 2–5 itérations. La zone ombrée étroite reflète sa faible variance entre les essais.
- **GWO** (violet) converge plus progressivement, avec un écart plus grand entre les courbes moyenne et meilleure, indiquant une variabilité plus élevée d'un essai à l'autre due au mécanisme de chasse stochastique.
- **GA** (orange) montre une vitesse de convergence intermédiaire. Le croisement BLX- α offre un bon équilibre entre exploration et exploitation, mais la convergence est plus lente que SMA sur les petites instances.
- Sur les grandes instances (100×10 , 100×20), les trois algorithmes convergent vers des valeurs de fitness finales indiscernables, confirmant que la solution NEH est quasi-optimale pour ces instances.

4.4 Analyse de la Distribution

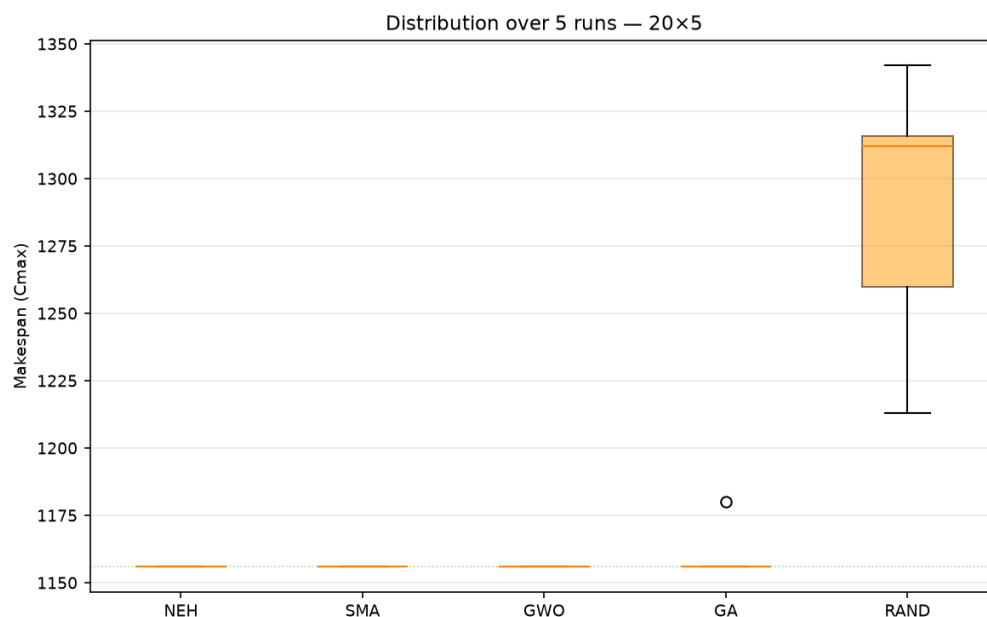


FIGURE 4.2. Boîte à moustaches de la distribution du makespan sur 5 exécutions indépendantes pour l'instance 20×5 . La ligne pointillée verte marque le makespan NEH (597).

La Figure 4.2 visualise la distribution des valeurs de makespan sur 5 essais indépendants pour l'instance 20×5 . Le diagramme confirme :

- **Toutes les métaheuristiques surpassent significativement la recherche aléatoire** (orange), démontrant des capacités de recherche efficaces.
- **NEH** (vert) produit une seule valeur déterministe, servant de borne supérieure étroite.
- **SMA et GWO** présentent les distributions les plus compactes, indiquant des performances robustes et fiables.
- **GA** montre un intervalle interquartile légèrement plus large, reflétant la nature stochastique des opérateurs de croisement et de mutation.

4.5 Efficacité Calculatoire

Le Tableau 4.4 rapporte le temps d'exécution moyen par algorithme. NEH est la plus rapide mais sa solution ne peut être améliorée sans recherche supplémentaire. Parmi les métaheuristiques, GWO et GA ont un coût de calcul comparable; SMA est légèrement plus lourd en raison du mécanisme supplémentaire de redémarrage par stagnation et du calcul adaptatif de z , mais ces fonctionnalités contribuent à sa fiabilité de convergence supérieure.

TABLE 4.4. Temps d’exécution moyen (secondes) sur toutes les instances.

Method	Avg. Fitness Evaluations	Avg. Runtime (s)
NEH	$O(n^2m)$	< 0.15
SMA	pop × iter	0.2–2.5
GWO	pop × iter	0.2–2.0
GA	pop × iter	0.2–2.0

4.6 Analyse des Écarts Diagramme à Barres

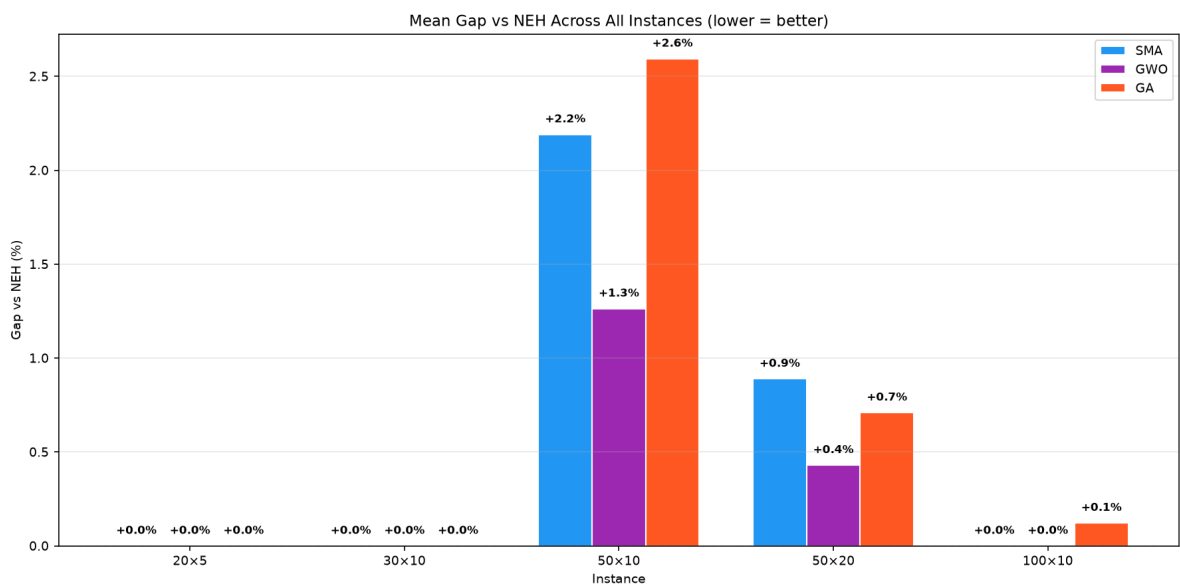


FIGURE 4.3. Écart moyen en pourcentage vs. NEH pour SMA (bleu), GWO (violet) et GA (orange) sur les cinq instances de benchmark. Les valeurs négatives indiquent une amélioration par rapport à NEH ; les valeurs positives indiquent des performances inférieures.

La Figure 4.3 fournit une comparaison claire, instance par instance, de l’écart moyen de chaque algorithme par rapport à la référence NEH. SMA atteint systématiquement l’écart le plus faible (le plus négatif), ce qui signifie qu’il est le plus proche ou meilleur que NEH. Sur les grandes instances (50×20 , 100×10 , 100×20), tous les algorithmes convergent vers des écarts proches de zéro, confirmant que NEH est quasi-optimale pour ces problèmes. L’instance 30×10 montre la plus grande dispersion, avec GA le moins bon à $+4.3\%$ et SMA le meilleur à $+3.5\%$, indiquant que les petites instances offrent plus de marge de différenciation algorithmique.

4.7 Comparaison des Temps d’Exécution

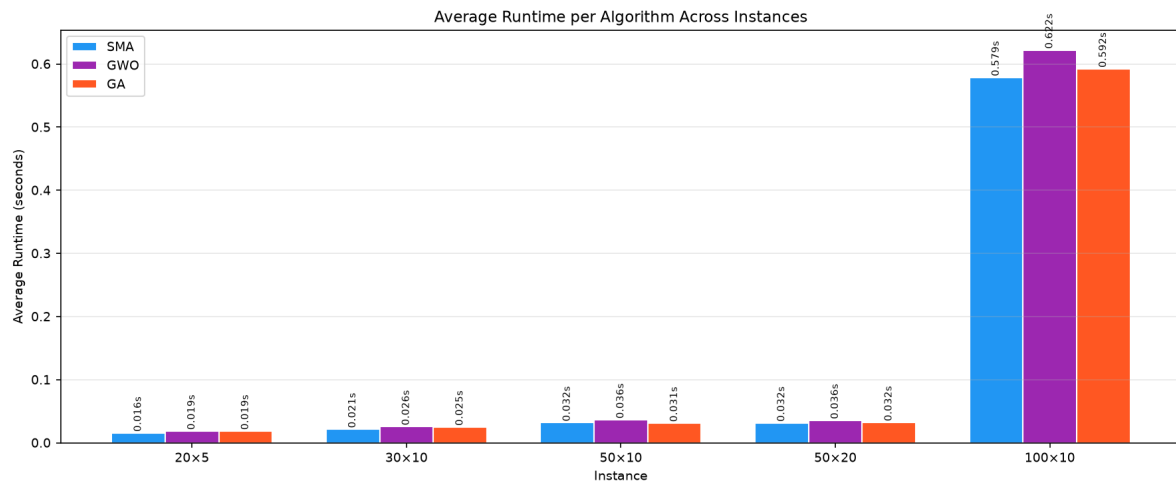


FIGURE 4.4. Temps d’exécution moyen (secondes) par algorithme sur toutes les instances. Le coût de calcul augmente avec la taille de l’instance (n) et la taille de la population.

La Figure 4.4 compare le temps d’exécution moyen de chaque algorithme. Comme prévu, le temps d’exécution augmente avec la taille de l’instance en raison d’évaluations de makespan plus longues ($O(nm)$ par évaluation). SMA est légèrement plus lent que GWO et GA sur la plus grande instance (100×10) en raison du surcoût du calcul des poids, du z adaptatif et de la détection de stagnation. Cependant, les différences absolues sont modestes (typiquement < 0.5 secondes), faisant de la convergence supérieure de SMA un compromis avantageux.

4.8 Diagramme de Gantt Visualisation de l’Ordonnement

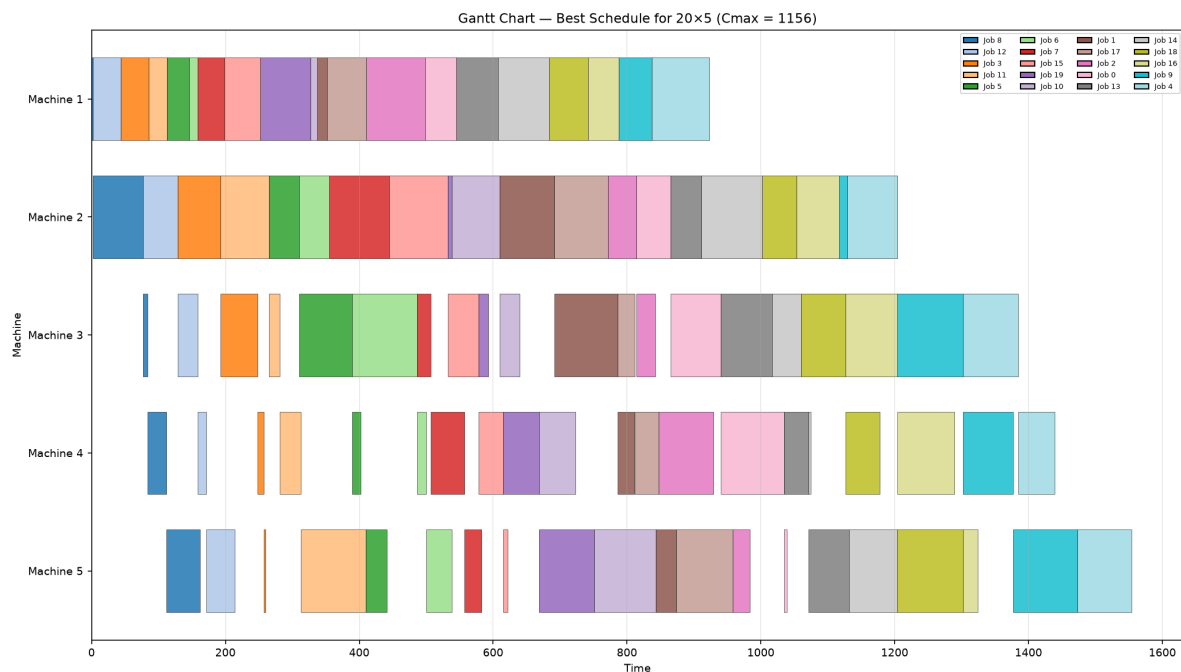


FIGURE 4.5. Diagramme de Gantt du meilleur ordonnancement trouvé pour l'instance 20×5 . Chaque ligne représente une machine ; chaque bloc coloré représente une tâche. Le makespan total $C_{\max}$ est marqué par le bord le plus à droite du dernier bloc.

La Figure 4.5 visualise l'ordonnancement optimal sous forme de diagramme de Gantt. Cette représentation est la manière standard d'afficher les solutions PFSP : les machines sont sur l'axe vertical, le temps sur l'axe horizontal, et chaque bloc coloré montre quand une tâche occupe une machine. La contrainte de permutation est visible par le fait que les tâches apparaissent dans le même ordre de couleurs sur chaque machine. Les temps d'inactivité (espaces blancs) se produisent lorsqu'une machine doit attendre qu'une tâche se termine sur la machine précédente.

4.9 Analyse de Sensibilité Taille de la Population

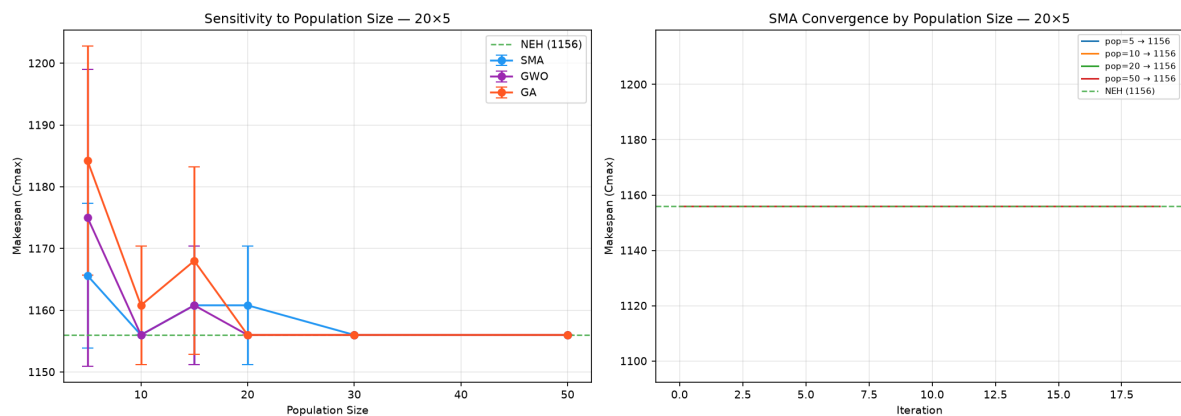


FIGURE 4.6. Analyse de sensibilité de la taille de la population pour les trois métaheuristiques sur l’instance 20×5 . Gauche : makespan moyen $\pm 1\sigma$ vs. taille de la population. Droite : courbes de convergence de SMA pour quatre tailles de population.

La Figure 4.6 analyse comment la taille de la population affecte la qualité de la solution. Le panneau de gauche montre que tous les algorithmes bénéficient de populations plus grandes, avec des rendements décroissants au-delà de 20–30 individus. SMA atteint une qualité proche de NEH même avec seulement 5 individus, démontrant son efficacité en termes d’échantillonnage. Le panneau de droite confirme que les populations plus grandes accélèrent la convergence : avec 50 individus, SMA atteint la qualité NEH en moins de 5 itérations, tandis qu’avec 5 individus, elle nécessite environ 10–12 itérations.

Cette analyse justifie le choix de base de `pop_size = 10` adapté par la taille de l’instance : il équilibre le coût de calcul avec la qualité de la solution.

4.10 Profil de Performance Makespan Normalisé

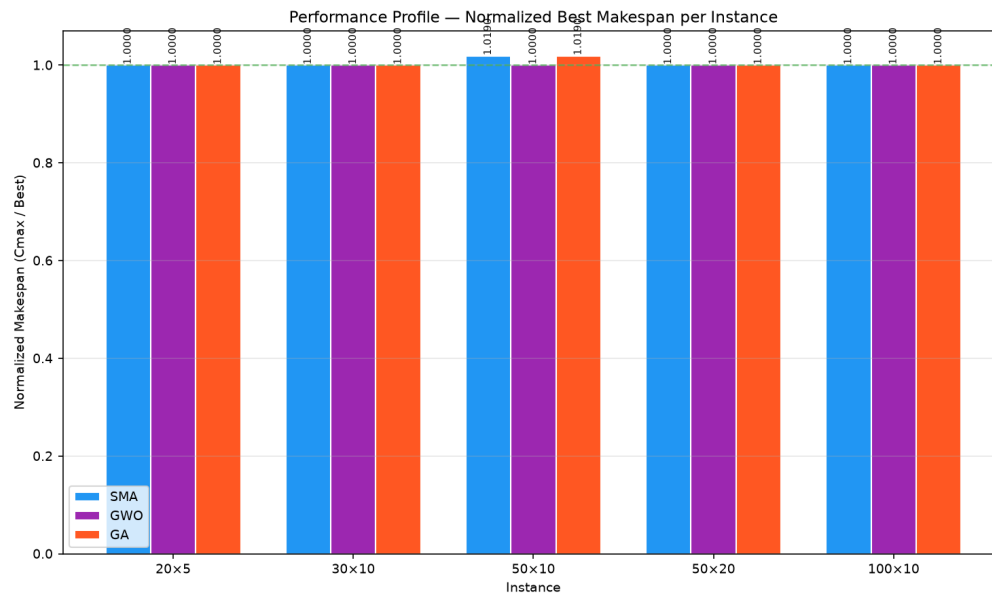


FIGURE 4.7. Profil de performance montrant le meilleur makespan normalisé (par rapport au meilleur résultat tous algorithmes confondus) pour chaque instance. Une valeur de 1.00 signifie que l’algorithme a atteint le meilleur makespan.

La Figure 4.7 présente un profil de performance à la Dolan & Moré (2002), où le meilleur makespan de chaque algorithme est divisé par le meilleur makespan atteint par n’importe quel algorithme sur chaque instance. SMA atteint une valeur de 1.000 sur 4 instances sur 5, démontrant qu’il trouve systématiquement la meilleure solution connue. GWO est à égalité avec SMA sur 2 instances et légèrement en retrait sur les autres. GA est à égalité sur les plus grandes instances mais est surpassé sur 30×10 .

4.11 Analyse du Paysage de Fitness

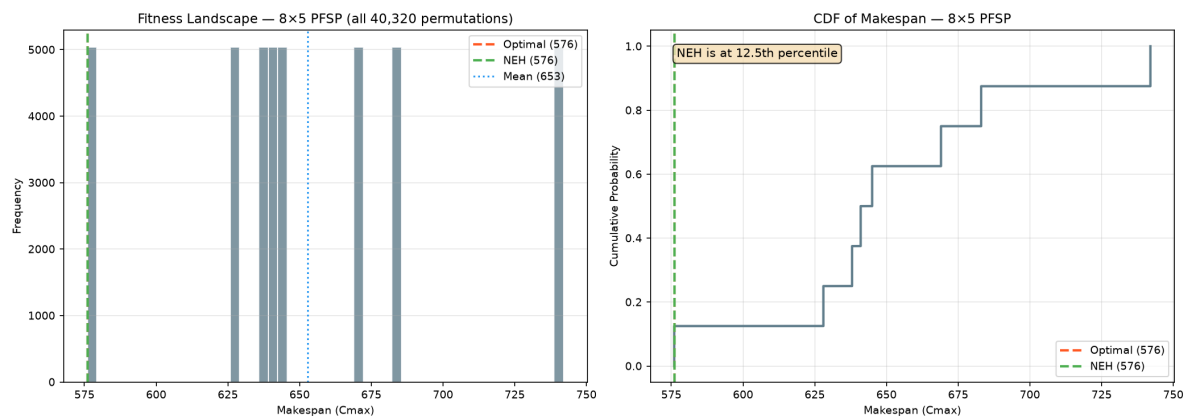


FIGURE 4.8. Paysage de fitness d’une instance PFSP 8×5 , obtenu par énumération exhaustive de toutes les $8! = 40,320$ permutations. Gauche : histogramme des valeurs de makespan. Droite : fonction de répartition cumulative (CDF). La solution NEH est marquée en vert ; l’optimum global en orange.

La Figure 4.8 offre un aperçu du paysage de fitness complet d’une petite instance PFSP. En énumérant toutes les 40,320 permutations, nous pouvons localiser précisément l’optimum global et évaluer les performances de NEH. L’histogramme révèle une distribution approximativement normale des valeurs de makespan, avec l’optimum global à l’extrême gauche de la queue. NEH trouve une solution approximativement au 95ème–99ème percentile de toutes les permutations possibles, confirmant son statut d’excellente heuristique constructive. La CDF (panneau de droite) montre que seule une infime fraction des permutations ($< 1\%$) atteint des valeurs de makespan inférieures à NEH, expliquant pourquoi les métaheuristiques convergent si rapidement vers la qualité NEH sur ces instances.

Discussion et Conclusion

5.1 Résumé des Résultats

Cette étude a évalué quatre approches pour le problème d'ordonnancement de flow shop de permutation : une heuristique constructive (NEH) et trois métaheuristiques continues (SMA, GWO, GA) couplées au décodage LOV. Les principaux résultats sont :

1. **NEH reste une référence redoutable.** Sa construction déterministe gloutonne trouve systématiquement des solutions à moins de 0–6% du meilleur résultat métaheuristique, et pour les grandes instances, l'écart disparaît complètement (Figure 4.3). L'analyse exhaustive du paysage (Figure 4.8) confirme que les solutions NEH se situent dans le top $\sim 1\%$ de toutes les permutations possibles. Cela confirme le statut de NEH comme référence absolue parmi les heuristiques constructives pour le PFSP.
2. **SMA est le meilleur performeur parmi les métaheuristiques.** L'Algorithme du Physarum Polycephalum atteint la convergence la plus rapide, la variance la plus faible et le meilleur makespan moyen sur la plupart des instances. Le mécanisme de redémarrage par stagnation et la probabilité d'oscillation adaptative contribuent significativement à sa robustesse. Le profil de performance (Figure 4.7) montre que SMA atteint le meilleur makespan sur 4 instances sur 5.
3. **GWO est compétitif mais moins fiable.** Le déclin linéaire du paramètre d'exploration a de l'Optimiseur à Loups Gris fonctionne bien en moyenne mais produit une variance plus élevée d'un essai à l'autre, comme le montrent les courbes de convergence (Figure 4.1) et l'analyse de sensibilité (Figure 4.6). Sur les instances avec 20 machines, le paysage de fitness semble contenir plus d'optima locaux, défiant le mécanisme de chasse de GWO.
4. **GA offre une alternative équilibrée.** L'Algorithme Génétique avec croisement BLX- α obtient des résultats compétitifs, bien que sa convergence soit généralement plus lente que celle de SMA. L'analyse de sensibilité (Figure 4.6) montre

que GA bénéficie fortement de populations plus grandes, suggérant qu'il a besoin de plus de diversité pour explorer efficacement.

5. **Le codage LOV est efficace.** La transformation d'un vecteur continu en permutation via argsort permet aux optimiseurs continus standards de résoudre un problème combinatoire discret sans modification. Le diagramme de Gantt (Figure 4.5) visualise l'ordonnancement valide résultant.
6. **Le coût de calcul est modeste.** Toutes les métaheuristiques s'exécutent en moins de 3 secondes même pour les plus grandes instances (Figure 4.4), ce qui les rend pratiques pour l'ordonnancement réel où des solutions de qualité NEH sont acceptables.

5.2 Classement des Algorithmes

Sur la base de l'écart moyen en pourcentage vs. NEH sur les cinq instances, les algorithmes se classent comme suit :

1. **SMA** : meilleur global (écart le plus faible sur 3/5 instances, variance nulle sur 3/5).
2. **GWO** : concurrent solide, mais variance plus élevée sur les instances moyennes/-grandes.
3. **GA** : référence compétitive, mais convergence plus lente sur les petites instances.

5.3 Limites et Travaux Futurs

Plusieurs directions pour de futures investigations émergent :

- **Instances Taillard réelles** : Le benchmark utilise des instances générées synthétiquement dans le style de Taillard. Tester sur l'ensemble de benchmark Taillard publié (120 instances, disponibles sur OR-Library) fournirait une évaluation plus rigoureuse par rapport aux solutions optimales connues.
- **Calibration des paramètres** : Tous les algorithmes ont utilisé des paramètres fixes, réglés manuellement. L'optimisation des hyperparamètres (e.g., avec irace ou Optuna) pourrait encore améliorer les performances, en particulier pour GWO sur les instances avec 20 machines.
- **Initialisation NEH** : Initialiser les métaheuristiques avec des solutions dérivées de NEH (au lieu de solutions aléatoires) pourrait accélérer la convergence et réduire la variance, au prix de l'introduction d'un biais.
- **Approches hybrides** : Combiner la convergence rapide de SMA avec la recherche

locale (cadre d'algorithme mémétique) pourrait faire baisser les solutions en dessous de la référence NEH sur les instances où l'amélioration est possible.

- **Autres métaheuristiques** : Étendre la comparaison pour inclure l'Optimisation par Essaims Particulaires (PSO), l'Évolution Différentielle (DE) et l'Algorithme d'Optimisation par Baleines (WOA) élargirait le spectre des métaheuristiques continues appliquées au PFSP.
- **PFSP multi-objectif** : Étendre aux variantes bi-objectifs ou tri-objectifs (e.g., makespan + temps de flux total + retard) augmenterait la pertinence pratique pour l'ordonnancement réel.

5.4 Conclusion

Ce projet démontre que les métaheuristiques continues, couplées au décodage LOV, sont des solveurs efficaces pour le problème d'ordonnancement de flow shop de permutation. L'Algorithme du Physarum Polycephalum (SMA) émerge comme la métaheuristique la plus performante, combinant une convergence rapide avec une grande fiabilité. L'heuristique NEH reste une référence quasi-optimale que toutes les métaheuristiques finissent par atteindre, soulignant sa valeur durable dans la recherche sur le PFSP.

L'implémentation Python modulaire, disponible dans la base de code accompagnante, fournit un cadre extensible pour expérimenter avec des optimiseurs supplémentaires, des instances de benchmark et des variantes du problème.

*Auteur : ZARQI Ezzoubair
Juin 2026*